arXiv:2611.08669v1 [cs.AI] 5 Nov 2026

CHAINFLOW: FEW-STEP LATENT FLOW MATCHING FOR SPECULATIVE DECODING

Selim Aktas [Q K] **Yusuf Erdem Erdem** [Q V] **Ahmed Zeer** [* Q V] **Himmet Toprak Kesgin** [V]
Muzaffer Kaan Yuce [V] **Mehmet Fatih Amasyali** [V]

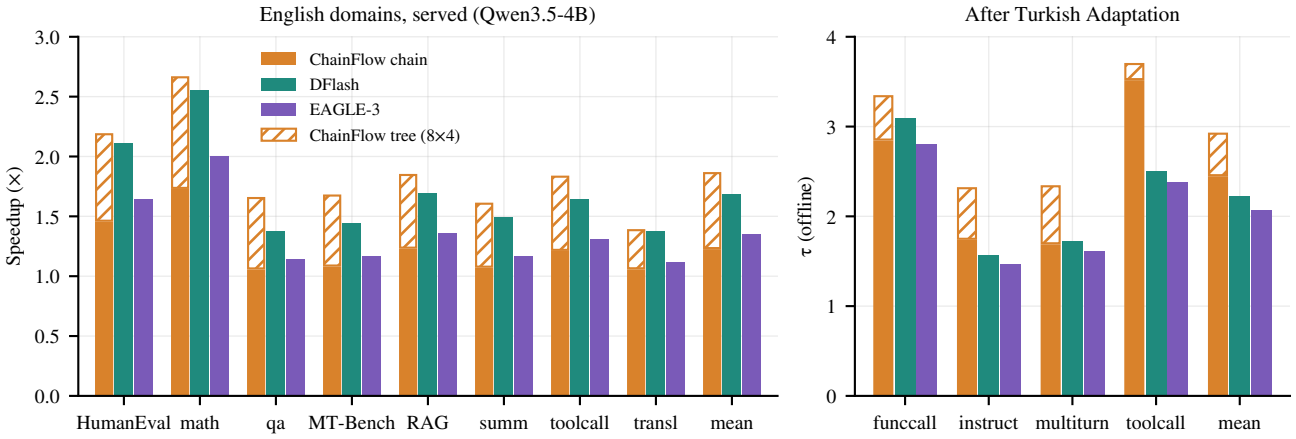

Figure 1. ChainFlow against EAGLE-3 and DFlash. Hatched: gain from 8×4 tree verification. (a) Served speedup over autoregressive decoding at batch one on English domains, ran on one NVIDIA B300 with vLLM 0.31.0 and greedy decoding. The tree beats both in every domain. (b) Acceptance length τ (offline) after Turkish adaptation; on average even the chain beats both, showing adaptability.

ABSTRACT

Speculative decoding has become a standard way to accelerate LLM inference: a lightweight drafter proposes tokens that the target verifies in one parallel pass. Existing drafters face trade-offs that limits their speedups: autoregressive drafters generate tokens one at a time, so drafting cost grows with draft length, while diffusion drafters generate tokens in parallel but predict each position independently, ignoring inter-token dependencies. We introduce ChainFlow, a new speculative decoding paradigm and the first flow-matching drafter conditioned on the target's hidden states. ChainFlow treats drafting as a short transport over hidden states, needing one or two flow steps instead of the many denoising steps of generation from noise. A variational autoencoder compresses these states so drafting cost barely grows with target size, lightweight feedback heads restore inter-token dependencies, and tree drafting raises acceptance. On Qwen3.5 models, at matched drafter size and training data, ChainFlow outperforms EAGLE-3 and DFlash in vLLM by 39% and 11%, respectively. These gains further widen as drafters are adapted to new languages sequentially. Our ablations cover drafter design and concurrent serving, and we open-source our code and drafter models.

[K]Department of Electronics Engineering, Koç University, Istanbul, Türkiye [V]Department of Computer Engineering, Yıldız Technical University, Istanbul, Türkiye. Correspondence to: Ahmed Zeer <ahmed.zeer@std.yildiz.edu.tr>.

[Q]Equal contribution. [*]Corresponding author.

Preprint.

1 INTRODUCTION

Large language models now sit behind a broad range of deployed systems, and the cost of running them is dominated by generation. Decoding is sequential: each token requires a full pass over the model's parameters, so inference is bound by memory bandwidth rather than compute (Shazeer, 2019). Long reasoning traces and agentic workloads sharpen the problem, because output length now sets latency.

Speculative decoding breaks part of this serialisation (Leviathan et al., 2023; Chen et al., 2023). A cheap drafter proposes future tokens; the target verifies them in one parallel pass and accepts the longest prefix consistent with its own distribution, leaving the output distribution unchanged. Per-token latency is then $L = (T_{\text{draft}} + T_{\text{verify}})/\tau$ for τ accepted tokens per cycle, and drafters sit at two poles of that trade-off. Autoregressive drafters such as EAGLE-3 (Li et al., 2025b) condition each token on the last and reach high τ, but pay $T_{\text{draft}} \propto \gamma$ and must stay shallow; drafting is reported to consume 20–40% of inference time (Liu et al., 2026). Parallel drafters (Chen et al., 2026a; Cheng et al., 2026) emit a whole block in one pass and can afford depth, but predict every position from the verified prefix alone. Their positions are marginal, so dependence between drafted tokens has to be restored after the fact.

Autoregressive drafters buy coherence with sequential cost; parallel drafters buy speed, but at a price we question. Each round begins from an uninformative seed, mask embeddings or noise, so the drafter rebuilds the block from scratch, and coherence is patched back afterwards by n-gram scores (Liu et al., 2026), recurrent correction heads (Cheng et al., 2026), or parent-conditioned trees (Ringel & Romano, 2026). Yet the target has already indicated where its next states lie, in its own recent hidden trajectory. A drafter seeded from that trajectory need not generate the block; it only has to *transport* an estimate onto the target's future states. Flow matching fits this view, since it constrains neither the source distribution nor the number of integration steps, and short transports need few (Lipman et al., 2023; 2024).

We introduce ChainFlow, a drafter that treats drafting as short-horizon transport on the target's hidden-state manifold. A variational autoencoder compresses the target's final hidden states to a latent space, and a conditional flow network transports an estimate of the next 8 latents, taken from the recent context, onto the target's future latents in one Euler step at serving time. Context states enter as cross-attention memory, so the drafter reads the target's internal representation without being embedded in it (Samragh et al., 2025). Decoded latents are projected by the target's frozen language modeling head (lm_head), so the drafter emits exact per-position categoricals and standard verification applies. A low-rank Markov bias and a path-conditioned residual then restore dependence on the sampled prefix without a second flow pass, and a beam search emits the drafts as a chain or a draft tree. One forward pass yields all 8 positions, and the flow never touches the backbone, so the drafter runs on an unmodified engine.

We train drafters for the 4B, 9B and 27B models of Qwen3.5 family. At batch one, ChainFlow reaches **3.23**× on English mathematics and **2.69**× on Turkish function calling at 27B (English per-domain results in Table 4). Drafters are trained first on English, then adapted on Turkish with data covering math, code, chat and open-ended sources. The English-only drafter is a net loss on Turkish; after adaptation Turkish improves sharply while English retains a speedup, so a drafter can be carried to a new language without retraining from scratch. German and French are then added in sequence, and replaying a small share of Turkish limits forgetting. Figure 1 summarises the comparison: served speedup on English domains (a), and acceptance length after Turkish adaptation (b). A context-derived source also makes the transport short: it trains faster than noise and stays accurate with a single integration step, where noise collapses, and one step is the fastest served configuration (§4.4). We also ladder concurrency: as with any speculative decoder, the gain narrows as the decode batch grows, and on one NVIDIA B300 the tree gains up to a batch of 4.

Our contributions are as follows.

- **Faster and more adaptable.** ChainFlow is 1.39× faster than EAGLE-3 and 1.11× faster than DFlash on English at matched drafter size, with the highest pooled speedup at 27B and the largest adaptation gains in Turkish, German and French.
- **Latent transport drafter.** Flow matching over a VAE latent of the target's hidden states, started from the extrapolated trajectory, drafts all positions in one or two steps.
- **Tree drafting at chain cost.** One pass yields every position's marginals, and token feedback turns them into chains or trees without re-running the drafter.

2 Related Work

Speculative decoding. Blockwise parallel decoding first proposed several positions against a single verifier pass (Stern et al., 2018), and rejection-sampling verification extended this to non-greedy generation while preserving the target distribution (Leviathan et al., 2023; Chen et al., 2023). Tree verification packs many candidate continuations into one attention pattern, so a rejected sibling does not invalidate a whole branch (Miao et al., 2024), and later work shapes that tree from drafter confidence (Li et al., 2024b). Training-free drafters built on prompt lookup or n-gram retrieval occupy the opposite extreme: drafting is nearly free, τ is low, and end-to-end gains stay modest (Saxena, 2023; Fu et al., 2024). A separate line treats the verification budget as a scheduling problem, pruning low-confidence proposals or adapting draft length to engine load and confidence (Cheng et al., 2026; Sadhukhan et al., 2025). ChainFlow inherits these, and our experiments include a decode-batch cutoff.

Evolution of drafter architectures. Early studies use a smaller model from the target's family, whose genera-

tion latency dominates the inference time (Leviathan et al., 2023; Chen et al., 2023). Medusa (Cai et al., 2024) and Hydra (Ankner et al., 2024) remove that second model with parameter-efficient heads on top-layer features, and the EAGLE line (Li et al., 2024a; 2025b) raises τ by predicting the next feature and reusing the target's head. While effective, these methods remain sequential in draft length. Block-parallel and diffusion drafters (An et al., 2025; Chen et al., 2026a; Liu et al., 2026) remove that dependence, permitting deeper drafters at constant latency, and DiffuSpec (Li et al., 2025a) shows the same with a pre-trained diffusion LM as the drafter. Because their per-position distributions are marginal, a further family restores intra-block dependency with lightweight sequential heads (Cheng et al., 2026; Huang et al., 2026) or branch-conditioned trees (Hu et al., 2026; Ringel & Romano, 2026).

Flow matching for sequence generation. Flow matching learns a velocity field transporting a source distribution to a data distribution along prescribed paths, without requiring an uninformative source (Lipman et al., 2023). Its discrete counterpart (Gat et al., 2024; Campbell et al., 2024) generalises masked diffusion via wider paths and couplings, placing diffusion language models (Nie et al., 2025) and block-diffusion drafters (Arriola et al., 2025) in a larger design space. Two recent results establish important properties: flow map language models (Lee et al., 2026) learn the endpoint map directly, generating in one or two steps rather than integrating a long trajectory, and LangFlow (Chen et al., 2026b) shows continuous flows over token embeddings match discrete formulations at comparable step budgets. Flow matching has also been proposed to replace or support autoregression, as a discrete or continuous flow over the full sequence (Wang et al., 2025; Shao et al., 2025); conversely, speculation accelerates sampling from flow models (Bajpai et al., 2026). No prior work uses a flow-matching transport as an autoregressive drafter, or conditions one on the target's hidden states, which, as we show, enables efficient few-step lossless drafting.

3 METHODOLOGY

ChainFlow follows the standard draft–verify cycle: the drafter proposes a block of continuations, the target scores all of them in one forward pass, and the longest accepted prefix is committed together with the target's own bonus token. Throughout, C denotes the number of context states the drafter reads, K the number of positions it drafts and N the number of integration steps; we use $C = 8$ and $K = 8$, train with $N = 2$, and serve with $N = 1$. The two differ by design: we train with $N = 2$ and ablate how many steps the trained transport needs at inference, finding that a single step suffices and is fastest when served (§4.4). Two problems shape the design. A block-parallel drafter produces all positions at once, so each position is a marginal over its predecessors and acceptance decays along the block; §3.1 addresses this with latent transport and post-hoc token feedback. And a longer block amortises verification only if the extra candidates are worth proposing and verifying; §3.2 covers candidate search and verification. Figure 2 walks through one cycle.

3.1 Latent Transport with Token Feedback

ChainFlow pays a fixed number of network evaluations for the whole block, but a block-parallel prediction is a marginal: position i cannot condition on the token sampled at $i-1$, so acceptance decays along the block whenever the context admits several continuations (Cheng et al., 2026). We attack both ends of this. The marginals themselves improve because the flow transports from an informative source rather than noise; the conditionals are restored after the flow by corrections that cost almost nothing.

Hidden-state compression. Following latent diffusion (Rombach et al., 2022) and latent flow matching (Dao et al., 2023; Esser et al., 2024), we use a VAE to separate representation compression from future-state modeling. Its bottleneck reduces the dimensionality of the flow's prediction problem, while the reconstruction objective encourages retention of information needed to recover the target's hidden states. A transformer VAE maps sequences of target hidden states to a lower-dimensional latent, $z = \mathrm{Enc}_\phi(h)$ and $\hat{h} = \mathrm{Dec}_\psi(z)$. Encoder and decoder are two-layer bidirectional transformers over windows of up to 16 states, so each latent also reflects its neighbours; dimensions are in Table 2. Because the flow's cost scales with the latent width Z rather than D, one network design serves every target size. The VAE is trained on cached hidden states with a small KL weight ($\beta = 3 \times 10^{-5}$), which discourages arbitrarily scaled latent representations while allowing faithful reconstruction (Rombach et al., 2022). It is then fine-tuned jointly with the drafter under a reconstruction term (§3.3); the drafter uses the posterior mean, so encoding is deterministic.

Flow matching with trajectory extrapolation. Let $c = (z_{t-C+1}, \ldots, z_t)$ be encoded context and $z_1 \in \mathbb{R}^{K \times Z}$ encoded future states. We use the rectified-flow path $z_s = (1-s)\, z_0 + s\, z_1$ for $s \in [0,1]$ (Lipman et al., 2023), training a velocity network v_θ to regress its constant velocity:

$$\mathcal{L}_{\text{flow}} = \mathbb{E}_s \left\| v_\theta(z_s, s, c) - (z_1 - z_0) \right\|^2. \tag{1}$$

Flow matching places no constraint on the source z_0, and we exploit this. Instead of Gaussian noise, draft position $i \in \{1, \ldots, K\}$ starts from a linear extrapolation of the last two context latents,

$$z_0^{(i)} = z_t + i\,(z_t - z_{t-1}), \tag{2}$$

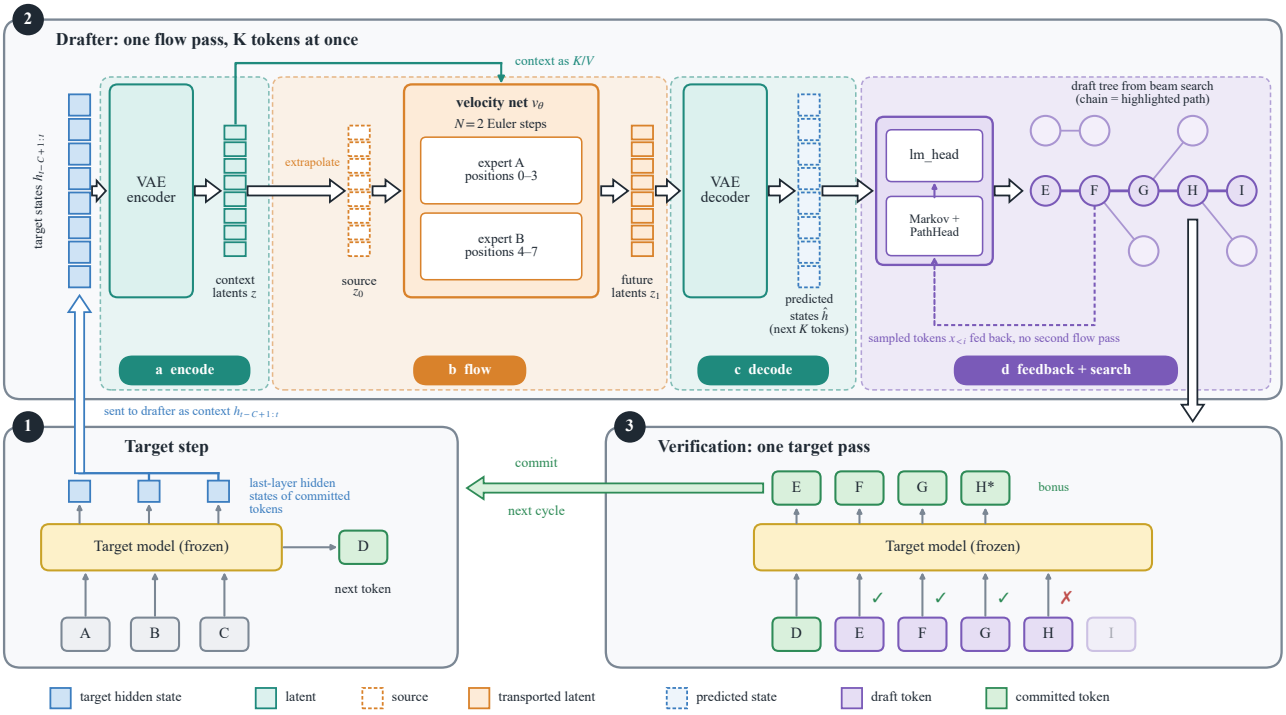

Figure 2. One draft–verify cycle of ChainFlow on a letter example. The drafter starts each position from the extrapolation $z_0^i = z_C + i\,(z_C - z_{C-1})$ of the context latents. The illustration uses N=2 Euler steps; served experiments use N=1. Here E, F, G are accepted and H is replaced by the target's own token H^*.

which we call *trajectory extrapolation*. Future states lie close to the recent trajectory, so the transport is short and nearly straight, and one or two Euler steps $z \leftarrow z + \frac{1}{N} v_\theta(z, s, c)$ suffice. Repeating the last latent and pure noise are the alternatives compared in §4.4.

Velocity network. v_θ is a pair of *chunk experts*: two separately parameterised networks, each responsible for one chunk of four consecutive draft positions (there is no routing; every expert always runs). Each expert is a stack of 8 pre-norm blocks with 8 heads and learned position and time embeddings. A block applies causal self-attention over the draft positions, cross-attention to the context latents c as keys and values, and a feed-forward layer of $6\times$ width. The context enters only as keys and values, so the drafter reads the target's representation without being embedded in it (Samragh et al., 2025). The two experts run concurrently, and a draft costs N network evaluations regardless of K. We ablate the number of experts in §4.4.

Post-hoc token feedback. After the last Euler step the decoder returns one hidden state per drafted position, $\hat{h}_{1:K} = \mathrm{Dec}_\psi(z)$, and a single matrix product with the frozen `lm_head` W turns all K of them into logits. Two lightweight corrections restore dependence on the chosen prefix after the flow:

$$p_\theta(x_i \mid x_{<i}) = \mathrm{softmax}\Big(W\big[\hat{h}_i + r_\omega(x_{i-p:i-1})\big] + U_2\,U_1[x_{i-1}]\Big). \tag{3}$$

The path head r_ω sums per-offset projections of the frozen token embeddings of the last $p = 8$ parent tokens and passes the sum through a small MLP with a zero-initialised output layer, so training starts from the marginal flow; it corrects the hidden state before the `lm_head`. The second term is a Markov head, as in Cheng et al. (2026): a rank-256 factorised first-order transition bias, $U_2\,U_1[x_{i-1}]$, that conditions each position on the immediately preceding sampled token. Neither term re-runs the flow or the VAE; only these corrections are evaluated per position. Training uses the target's tokens as parents; inference uses the tokens chosen along each beam.

3.2 Candidate Search and Verification

A longer draft block amortises the fixed verification cost over more accepted tokens, but only if the extra candidates are worth proposing and verifying. This subsection decides what to propose, how to search it, and how it is verified.

Two-pass candidate head. Scoring every candidate against the full `lm_head` would dominate draft time, so the head is restricted to a shortlist of the 62,642 tokens that occur in the training corpus. One matrix product of $\hat{h}_{1:K}$ against the shortlist selects a shared set of $M = 8192$ candidate ids, and Eq. 3 is evaluated only over those rows. This affects only what the drafter can propose: a missing token lowers acceptance but cannot change the target's output.

Chain and tree emission. A beam search over Eq. 3 emits the candidates as a chain or a tree (Figure 3). A chain keeps only the best path, of length 3. A tree of shape $W_b \times D$ is built level by level: at each depth, the nodes of the previous level are expanded with their top candidate tokens, all resulting children are ranked by cumulative log-probability, and the best W_b are kept, each with a pointer to its parent. Our headline 8×4 tree (W_b=8, D=4) therefore has 32 draft nodes, and a rejected sibling does not invalidate the round. Tree shape trades verification cost against acceptance, so we vary W_b and D in an ablation (§4.4). The target verifies a tree in one pass by flattening its nodes into a sequence under an ancestor mask: each node attends only to the committed prefix and its ancestors, so every root-to-node path is scored as a separate chain. Chains are verified by standard rejection; tree verification needs forked recurrent state for Qwen3.5's linear-attention layers and supports greedy decoding only. In both modes a token is emitted only if the target's own logits select it, up to fp16 ties.

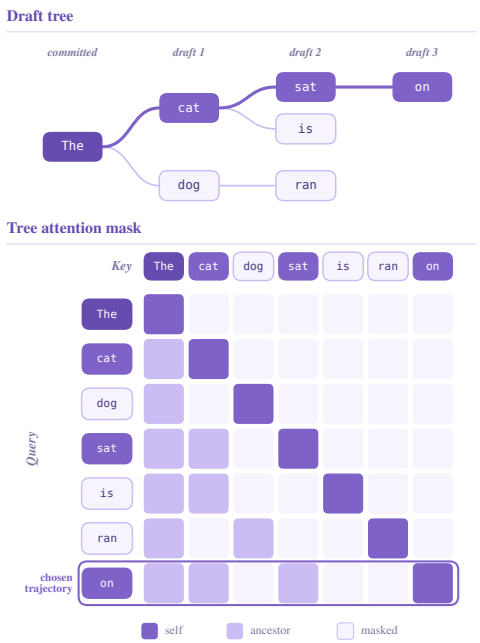

Figure 3. Tree drafting and attention mask (illustrative). Each draft token attends only to itself and its ancestors, so all branches are verified in one target pass.

3.3 Training

Data. The target generates a greedy response for every training prompt (Li et al., 2025b; Chen et al., 2026a), and one forward pass caches its final hidden states. Each example is a window anchored at a response position t: the states $h_{t-C+1:t}$ form the context, and the next K hidden states and tokens $x^\star_{1:K}$ are the targets.

Objective. The flow, path head, Markov head and VAE are trained end-to-end with

$$\mathcal{L} = 0.2\,\mathcal{L}_{\text{flow}} + 0.25\,\mathcal{L}_{\text{hid}} + 0.3\,\mathcal{L}_{\text{CE}} + 0.3\,\mathcal{L}_{\text{cov}} + 0.15\,\mathcal{L}_{\text{acc}} + 0.5\,\mathcal{L}_{\text{rec}}. \tag{4}$$

$\mathcal{L}_{\text{hid}}$ is the relative squared error against the target's hidden states, with and without the path correction, evaluated on predictions integrated through the full N-step trajectory so it sees the trajectory used at inference. $\mathcal{L}_{\text{CE}}$ is the cross-entropy of Eq. 3 plus a $0.2\times$ term on the marginal logits, which keeps the uncorrected flow decodable. $\mathcal{L}_{\text{cov}}$ is a hinge that pushes the true token into the top $b = 8$ logits, preparing the drafter for tree expansion. $\mathcal{L}_{\text{rec}}$ is the VAE reconstruction error. Finally,

$$\mathcal{L}_{\text{acc}} = -\sum_i \gamma^i \prod_{j \le i} p_\theta(x^\star_j \mid x^\star_{<j}) \tag{5}$$

with $\gamma = 0.8$ is a differentiable surrogate for the expected accepted length.

Optimization. All modules train with AdamW at a constant 5×10^{-4} for 15 epochs, with batches of about 6K windows and the VAE at $1/20$ of the drafter's learning rate. Adaptation to a new language warm-starts from the English drafter and continues with the same objective at $1\text{–}1.5 \times 10^{-4}$ for 2–6 epochs. Languages are added in sequence (Turkish, then German, then French), optionally mixing in a share of earlier-language rows as replay (§4.3).

4 Experiments

4.1 Experimental Setup

Target and draft models. We train one drafter for each Qwen3.5 (Qwen Team, 2026) target (Table 2). All drafters read $C = 8$ context states, draft $K = 8$ positions, are trained with $N = 2$ Euler steps, and use 8 blocks with an FFN multiplier of 6; only the latent width changes with the target. Drafts are served as chains of 3 tokens or as 8×4 trees, with a single Euler step (§4.4).

Training data. Drafters are first trained on 45.9K English prompts covering mathematics and science, code, dialogue, instruction following, summarisation, translation and creative writing. They are then adapted in sequence,

Table 1. English results at Qwen3.5-4B, batch one: acceptance length τ and speedup V over autoregressive decoding by domain and pooled mean. C denotes chain drafting (K=3); T1 and T2 denote 8×4 and 5×7 trees (nodes per depth $\times$ depth), respectively, with T1 as our headline configuration. Each drafter has ∼0.52B parameters. EAGLE-3 and DFlash draft K=4 tokens per round. Evaluated on NVIDIA B300 with vLLM 0.31.0 and greedy decoding.

Method	Code		Math		QA		MT		RAG		Summ.		Tool		Transl.		Mean	
	τ	V	τ	V	τ	V	τ	V	τ	V	τ	V	τ	V	τ	V	τ	V
EAGLE-3	3.09	1.64×	3.80	2.00×	2.09	1.14×	2.12	1.17×	2.48	1.36×	2.10	1.17×	2.41	1.31×	2.30	1.11×	2.49	1.35×
DFlash	3.02	2.11×	3.68	2.55×	1.89	1.38×	1.98	1.44×	2.34	1.69×	2.03	1.49×	2.29	1.64×	2.24	1.38×	2.35	1.69×
ChainFlow C	2.43	1.45×	2.91	1.73×	1.72	1.06×	1.76	1.08×	1.99	1.23×	1.72	1.07×	1.98	1.21×	1.85	1.06×	2.01	1.23×
ChainFlow T1	3.31	**2.19**×	4.07	2.67×	**2.43**	**1.66**×	**2.44**	**1.68**×	**2.73**	**1.85**×	**2.32**	**1.61**×	**2.74**	**1.84**×	**2.58**	**1.39**×	**2.78**	**1.87**×
ChainFlow T2	**3.41**	2.14×	**4.51**	**2.78**×	2.31	1.51×	2.33	1.53×	2.65	1.72×	2.23	1.48×	2.64	1.69×	2.46	1.32×	2.73	1.75×

Table 2. Drafters. Parameter counts exclude the target's shared, frozen embedding and `lm_head`.

Target	Hidden D	Latent Z	Drafter params
Qwen3.5-4B	2560	640	529M
Qwen3.5-9B	4096	1024	1.16B
Qwen3.5-27B	5120	1024	1.64B

without English replay, to Turkish, German and French on domains covering instruction following, function and tool calling, multi-turn dialogue and retrieval-augmented generation. Training responses are the target's own greedy continuations. Appendix A lists every source.

Benchmarks. English uses the eight RedHatAI speculator benchmark domains: code (HumanEval (Chen et al., 2021)), mathematical reasoning, question answering, MT-Bench, retrieval-augmented generation, summarisation, tool calling and translation. Other languages use held-out prompts of their training sources. Generations are limited to 256 tokens, and tokens are pooled across domains.

Protocol and metrics. All methods run on one NVIDIA B300 GPU with vLLM 0.31.0 (Kwon et al., 2023), greedy decoding and batch size one unless stated, with asynchronous scheduling for every arm including the baseline. Requests stop at their natural end of sequence or at 256 new tokens, and each served number pools two or three server launches. The ChainFlow drafter is implemented as a custom proposer in vLLM.

We report *speedup* in output tokens per second over autoregressive decoding and *acceptance length* τ, the tokens produced per round including the bonus token. Cross-method multilingual and per-position comparisons use τ measured offline on cached target states, with each drafter given the information the engine gives it when serving.

Comparisons. We compare against EAGLE-3 (Li et al., 2025b) and DFlash (Chen et al., 2026a), retrained on the same data with drafters matched in size to ours at 4B and served on the same engine, drafting K=4 tokens per round, the depth of our headline tree. We additionally evaluate scaling across the 4B, 9B, and 27B targets (Table 3) and examine drafter design choices through development sweeps (§4.4). These cover token feedback, expert partitioning, source initialization, integration steps, network capacity, and draft tree construction.

4.2 Main Results

Comparisons. Pooled over the eight domains, the ChainFlow 8×4 tree is 1.11× faster than DFlash and 1.39× faster than EAGLE-3 (Table 1), and it leads DFlash in every domain, by the least on translation (1.39× against 1.38×). The chain, which needs no engine change, reaches 1.23× and trails both baselines; tree verification lifts τ from 2.01 to 2.78. Gains follow how predictable a domain is: code and mathematics, whose outputs are structured and partly copied from the prompt, gain most, and translation, whose output language differs from the context, least.

Table 3. Scaling with target size. Flow and AR latencies are per draft and decoding step (ms), respectively. τ is measured on a fixed 8×4 tree; chain uses $K = 3$.

Target	Flow ms	AR step ms	τ	Chain	Tree
4B	0.30	3.1	2.78	1.23×	1.87×
9B	0.38	4.2	2.66	1.30×	1.78×
27B	0.38	11.2	2.73	1.61×	2.12×

Scaling with target size. Tree acceptance stays between 2.66 and 2.78 from 4B to 27B, while flow integration takes 0.30–0.38 ms and the autoregressive step grows from 3.1 to 11.2 ms. Chain speedup rises from 1.23× to 1.30× and 1.61×; tree speedup is 1.87×, 1.78× and 2.12×, peaking at 27B (Table 3). Tree drafting is profitable on translation at every size, with speedups of 1.39×, 1.42× and 1.74×.

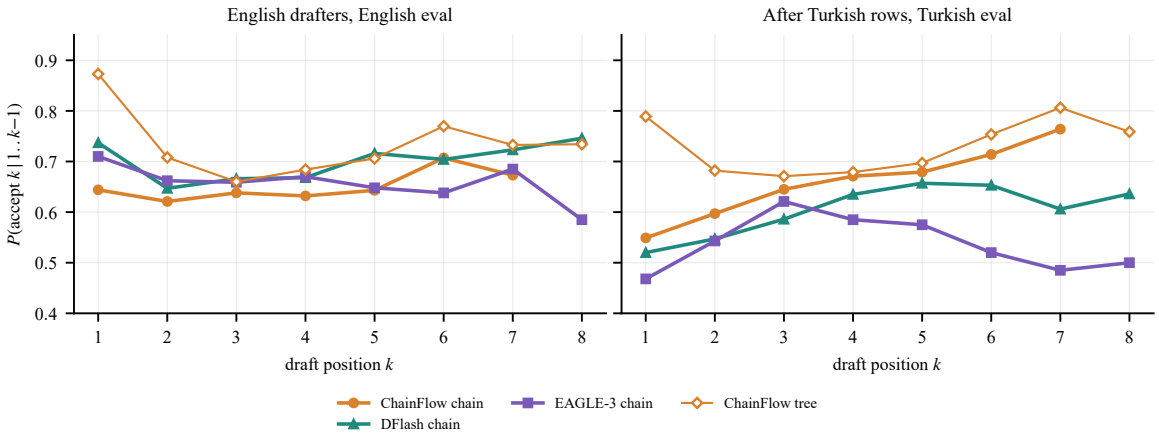

Figure 4. Conditional acceptance at each draft position, given that all earlier positions were accepted (offline, 4B). Left: English drafters on English. Right: after Turkish adaptation, on Turkish. Chain curves end at position 7; the tree is 8×4.

Table 4 breaks the tree speedup down by domain. The ordering of domains is the same at every size, with math and code highest and translation lowest. Speedup in 27B target leads in every domain, peaking at $3.23\times$ on math.

Table 4. Per-domain tree speedup (8×4) by target size in English.

Target	Code	Math	QA	MT	RAG	Summ.	Tool	Transl.
4B	2.19	2.67	1.66	1.68	1.85	1.61	1.84	1.39
9B	2.13	2.67	1.52	1.54	1.73	1.52	1.83	1.42
27B	**2.51**	**3.23**	**1.82**	**1.87**	**2.09**	**1.79**	**2.09**	**1.74**

Acceptance along the block. Figure 4 shows the probability of accepting each draft position given all earlier ones were accepted. Parallel drafters do not decay along the block: acceptance is flat or rising for ChainFlow and DFlash, whereas sequential EAGLE-3 declines, markedly so on Turkish. On English the chain's curve is flat but slightly below the baselines, so its lower τ reflects a uniformly lower per-position level, not a deeper failure. The tree keeps ChainFlow at or above both baselines at nearly every position, most clearly at the first. After Turkish adaptation both ChainFlow variants lead everywhere, and the chain's acceptance rises along the block, consistent with later positions conditioning on a longer accepted prefix.

4.3 Multilingual Adaptation

Adapting to new languages. ChainFlow gains most in every new language under sequential adaptation (Figure 5), while English performance drifts down similarly for all three methods, so this drop does not distinguish them. Served without adaptation, an English drafter is a net loss on Turkish at 4B and 9B; adaptation yields tree speedups of $1.81\times$, $1.87\times$ and $2.25\times$ at 4B, 9B and 27B, respectively.

Retaining earlier languages. This plasticity has a cost. Without replay, adding German erases most of the Turkish gain, a loss five to nine times larger than that of the baselines. Mixing a fifth of Turkish rows into each later training stage removes nearly all of this forgetting while leaving the new-language gain intact, so ChainFlow can adapt to a new language without giving up the earlier ones.

Draft vocabulary. Part of the lead comes from the candidate head. Our per-language shortlist covers nearly all Turkish evaluation tokens, whereas the baselines' frozen English vocabulary covers only about three in five. On French, where this gap is smallest, our gain is still the largest.

4.4 Drafter Design

ChainFlow combines many methods, and each component affects the end-to-end speedup differently. To understand these effects, we isolate four design decisions: token feedback, the number of experts, the source distribution, and expert width. We measure the effect of each change separately in each setting, and then combine the best-performing choices into the final drafter. All experiments use the Qwen3.5-0.8B target model, $K = 4$ draft positions, and the GSM8K dataset.

Table 5. Token feedback. acc@i: acceptance at draft position i; speedup measured live.

Drafter	acc@1	acc@4	τ	Speedup
Flow, context only	0.96	0.44	2.68	$0.96\times$
+ deep-position loss	0.96	0.44	2.89	$0.99\times$
+ Markov head	0.95	0.46	2.93	$1.01\times$
Autoregressive	0.99	0.57	3.21	$1.00\times$

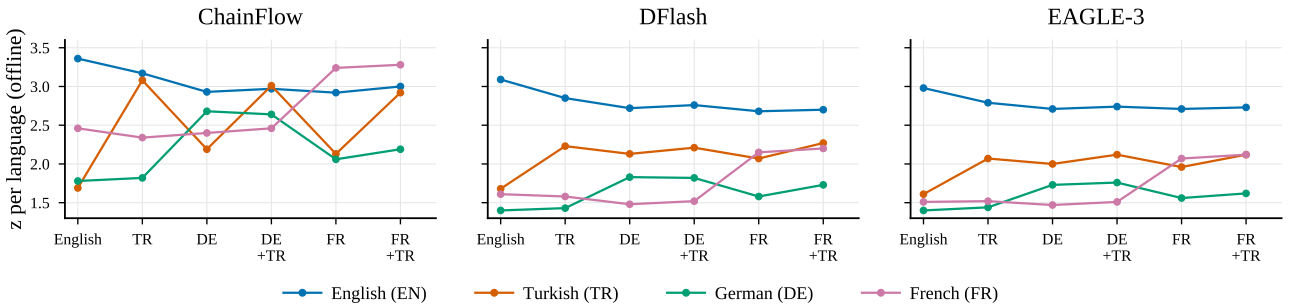

Figure 5. Effect of language adaptation on offline decoding speedup (z) for ChainFlow (tree mode), DFlash, and EAGLE-3. Adapting to new languages improves their decoding speedups, with ChainFlow achieving the largest gains, while English performance declines slightly across all three methods.

Table 6. Splitting the draft block across experts. Plateau: validation expected acceptance relative to 2×4. Values are approximate.

Experts × positions	Plateau	Epochs to 90%
2×4 (ChainFlow)	1.00	≈85
4×2	≈0.99	≈80

Token feedback. Drafters that predict every position from the context alone accept the first position almost always, later ones rarely, and run slower than autoregressive decoding (Table 5). Loss reweighting, more experts, other sources and more capacity all left this ceiling in place (Tables 6 and 7, Figure 7). Feeding back the chosen token lifts it: the Markov head closes part of the gap to an autoregressive drafter within one flow pass, and the path head extends the feedback to the full prefix and to the hidden state.

Experts. Increasing the number of experts from two to four does not raise validation acceptance beyond the plateau already reached with two (Table 6). Two experts are therefore sufficient to cover our eight-position draft, and ChainFlow uses two experts, each handling four positions, which avoids the cost of additional experts.

Table 7. Source distribution. Left: training epochs to 90% of the validation plateau. Right: inference-time Euler steps for 4B drafters trained at $N = 2$ (offline, English chain).

Source z_0	Dev. sweep epochs to 90%	4B, τ N=1	N=2	N=4
Noise	≈57	1.97	2.55	2.47
Repeat last	≈38	2.58	2.66	2.42
Extrapolation	≈20	2.49	2.53	2.50

Source distribution and integration steps. An informative source shortens the transport. All three sources reach the same plateau, but context-derived ones get there much faster (Figure 6), and at inference they survive a single Euler step, where noise collapses (Table 7). Extrapolation and repeating the last latent do not separate at this scale. Served, a second Euler step raises τ from 2.78 to 2.83 but reduces throughput by 5% (611.7 to 580.1 tokens/s), so the served configuration uses one step.

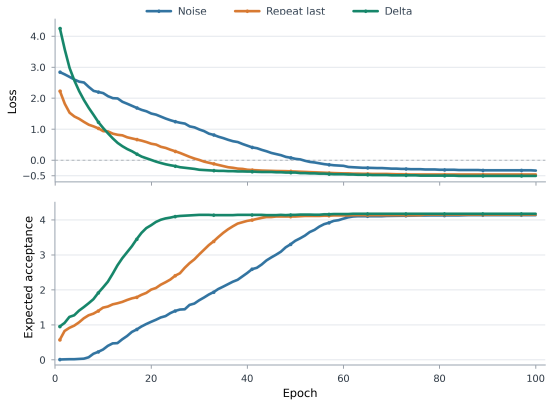

Figure 6. Training loss (top) and validation expected acceptance (bottom) across epochs for three initializations: noise, last-latent repetition, and trajectory extrapolation (Delta).

Capacity. We vary expert width, the FFN multiplier and context length in separate development sweeps with MLP experts (Figure 7). Increasing width improves acceptance, while increasing the FFN multiplier offers little gain. Longer contexts reduce acceptance in these sweeps, motivating our choice of $C = 8$ context states. No capacity setting lifted the ceiling that token feedback removes.

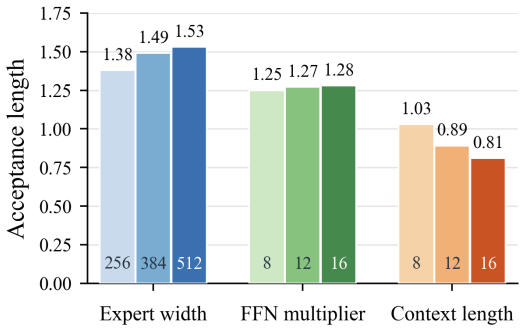

Figure 7. Capacity and context sweeps for flow with MLP experts.

Draft tree construction. A tree spends target compute on extra candidates, so its shape sets how acceptance trades against cost. Unlike the sweeps above, this ablation is served end to end on the 4B target at batch one: we vary the nodes kept per depth W_b and the depth D while holding the drafter fixed (Table 8). Acceptance grows with both width and depth, but every extra node must be searched and verified, so throughput falls. Beyond 8×4, the gain in τ is smaller than the added cost, and 8×4 is the fastest tree. Narrow, deep trees fare worst: 5×7 has both the lowest τ and the lowest throughput, so depth does not pay without width. A compact tree therefore beats larger ones, and we use 8×4 throughout.

Table 8. Tree shape ($W_b \times D$: nodes kept per depth $\times$ depth) at Qwen3.5-4B, batch one, eight English domains. Positions: tokens verified per round, including the committed token. Speedup is relative to plain decoding (327 tokens/s). The headline 8×4 tree is best.

Tree Shape	Positions	Tokens/s	τ	Speedup
8×4	33	**610.9**	2.82	**1.87**×
8×5	41	606.2	2.84	1.85×
8×6	49	596.4	2.87	1.83×
12×4	49	593.6	2.88	1.82×
6×5	31	593.4	2.74	1.82×
16×4	65	589.0	**2.96**	1.80×
5×7	36	572.8	2.73	1.75×

4.5 Serving and Vocabulary Shortlisting

Serving under concurrency. The gain narrows as concurrency grows (Table 9): every request sends the tree's 33 positions through a target that is no longer memory-bound, while acceptance stays between 2.77 and 2.85. The tree gains up to 4 concurrent requests, so speculation with the tree pays off only up to that decode batch, and beyond it plain autoregressive decoding is faster. EAGLE-3 and DFlash, which verify only $K+1$ positions per request, keep a gain at every level.

Table 9. Speedup under concurrency at Qwen3.5-4B: output tokens per second relative to autoregressive decoding at the same number of concurrent requests.

	Concurrent requests					
Method	1	4	8	16	64	128
EAGLE-3	1.33	1.26	1.23	1.20	1.20	1.06
DFlash	1.66	1.48	1.39	1.33	1.32	1.14
ChainFlow	1.85	1.12	0.94	0.81	0.63	0.45

Vocabulary shortlisting. On the eight-domain 4B benchmark (8×4 tree, batch one), restricting the candidate head to the shortlist leaves throughput and acceptance nearly unchanged (540.4 vs 541.3 tokens/s, τ 2.778); two-pass rescoring of the top $M = 8192$ candidates raises throughput to 611.7 tokens/s (13%) at τ 2.775. Reducing the budget to $M = 1024$ adds 1.1% throughput at τ 2.764. Conversely, using $M = 8192$ instead of 1024 increases acceptance length by 0.011 tokens per round (0.4%).

5 Conclusion

This study presented ChainFlow, a speculative drafter built on continuous flow-matching over the target's hidden states. A single Euler step yields the highest served throughput, since further steps add little acceptance for their drafting time. Because a variational autoencoder sets flow cost by its latent width, flow integration takes 0.30–0.38 ms from 4B to 27B while the autoregressive step grows from 3.1 to 11.2 ms, and the pooled speedup peaks at 2.12× at 27B. With tree drafting, ChainFlow outperforms EAGLE-3 and DFlash by 39% and 11% at matched drafter size and training data, and ablations on tree drafting show that a compact tree can achieve higher speedups than larger, deeper ones. Adapted in sequence to Turkish, German and French, ChainFlow leads both baselines in every language, and the gap widens as new languages are added.

The broader contribution of this work is a formulation of speculative drafting as transport over the target's own representations, together with evidence that this formulation holds up across model scales and languages. The formulation exposes design axes specific to flow-based drafting: where the transport starts, the latent space it operates in, and the feedback that restores token dependencies. Load-aware verification and other model families are natural next steps.

Limitations

Like other speculative decoding methods, ChainFlow gains less at higher concurrency, since larger batches leave less idle compute for verifying drafts. Each serving configuration uses a fixed draft length and tree topology, and while

the decode-batch cutoff disables speculation under load, it does not adjust how many tokens are proposed. Budget-aware methods such as BASTION (Oh et al., 2026) and EAGLE-4 (EAGLE-4 Team, 2026) adapt draft trees to confidence or execution cost, and adding such adaptation to ChainFlow is future work. The VAE adds a second trade-off. A compact latent space keeps the flow cheap to train and run, but may lose information needed to reconstruct the target's hidden states, which caps acceptance. A larger latent could help at the cost of slower drafting, a trade-off we have not characterized.

More thorough evaluation would strengthen our conclusions. All experiments use Qwen3.5 targets, whose hybrid attention required engine-specific state handling, so how well ChainFlow transfers to other model families remains open. Several ablations were run only on a 0.8B target for cost. Our drafters may be undertrained: training on more data would show how the different approaches scale, and pairing larger targets with larger drafters is another open direction.

Tree verification requires a patched engine and greedy decoding. In the B300 runs, 4–13% of prompts produce different outputs from plain greedy decoding across target sizes and languages, with differences attributed to fp16 near-ties during batched verification. EAGLE-3 and DFlash also produce differing outputs on the same evaluation prompts. These served measurements therefore do not establish bitwise equivalence to plain decoding.

REFERENCES

Altinok, D. InstrucTurca: A diverse instructional content dataset for turkish. Hugging Face dataset, 2024. URL https://huggingface.co/datasets/turkish-nlp-suite/InstrucTurca.

An, Z., Bai, H., Liu, Z., Li, D., and Barsoum, E. Pard: Accelerating llm inference with low-cost parallel draft model adaptation. *arXiv preprint arXiv:2504.18583*, 2025.

Ankner, Z., Parthasarathy, R., Nrusimha, A., Rinard, C., Ragan-Kelley, J., and Brandon, W. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.

Arriola, M., Gokaslan, A., Chiu, J. T., Yang, Z., Qi, Z., Han, J., Sahoo, S. S., and Kuleshov, V. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations (ICLR)*, 2025.

Ataşoğlu, A. Turkish tool calling 10k. Hugging Face dataset, 2025. URL https://huggingface.co/datasets/atasoglu/turkish-tool-calling-10k.

Ataşoğlu, A. Turkish tool calling dataset collection. Hugging Face dataset collection, n.d. URL https://huggingface.co/collections/atasoglu/turkish-tool-calling-dataset.

Auras, D., Harries, J., and Pluster, B. GermanRAG. DiscoResearch dataset, n.d. URL https://huggingface.co/datasets/DiscoResearch/germanrag.

Bajpai, D. J., Agarwal, S., Saxena, A., Kulkarni, K., Mitra, S., and Hanawal, M. K. Flowcast: Trajectory forecasting for scalable zero-cost speculative flow matching. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=gv8R88PVEG.

Bayram, A. Turkish multi turn dialog dataset. Hugging Face dataset, 2026. URL https://huggingface.co/datasets/magibu/turkish-multi-turn-dialog-dataset.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, pp. 5209–5235, 2024.

Campbell, A., Yim, J., Barzilay, R., Rainforth, T., and Jaakkola, T. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *International Conference on Machine Learning (ICML)*, 2024.

CausalLM. GPT-4 Self-Instruct German. Hugging Face dataset, n.d. URL https://huggingface.co/datasets/CausalLM/GPT-4-Self-Instruct-German.

Chaudhary, S. Code alpaca: An instruction-following LLaMA model for code generation. GitHub repository, 2023. URL https://github.com/sahil280114/codealpaca.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Chen, J., Liang, Y., and Liu, Z. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026a.

Chen, M., Tworek, J., Jun, H., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

Chen, Y., Liang, C., Sui, H., Guo, R., Cheng, C., You, J., and Liu, G. Langflow: Continuous diffusion rivals discrete in language modeling. *arXiv preprint arXiv:2604.11748*, 2026b.

Cheng, X., Yu, X., Shao, C., Li, J., Xiong, Y., et al. Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation. *arXiv preprint arXiv:2607.05147*, 2026.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

Conover, M., Hayes, M., Mathur, A., Xie, J., Wan, J., Shah, S., Ghodsi, A., Wendell, P., Zaharia, M., and Xin, R. Free dolly: Introducing the world's first truly open instruction-tuned LLM, 2023. URL https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm.

Dao, Q., Phung, H., Nguyen, B., and Tran, A. Flow matching in latent space. *arXiv preprint arXiv:2307.08698*, 2023. URL https://arxiv.org/abs/2307.08698.

Ding, N., Chen, Y., Xu, B., Qin, Y., Zheng, Z., Hu, S., Liu, Z., Sun, M., and Zhou, B. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023. URL https://arxiv.org/abs/2305.14233.

EAGLE-4 Team. EAGLE-4: One flexible drafter for all batch sizes and hardware, 2026. URL https://github.com/eagle-hq/EAGLE-4/blob/main/EAGLE4_Paper.pdf. Preprint.

Esser, P., Kulal, S., Blattmann, A., Entezari, R., Müller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., Boesel, F., Podell, D., Dockhorn, T., English, Z., Lacey, K., Goodwin, A., Marek, Y., and Rombach, R. Scaling rectified flow transformers for high-resolution image synthesis. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2403.03206.

Fan, A., Lewis, M., and Dauphin, Y. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 889–898. Association for Computational Linguistics, 2018. doi: 10.18653/v1/P18-1082. URL https://aclanthology.org/P18-1082/.

FreedomIntelligence. Alpaca-GPT4-Deutsch. Hugging Face dataset, 2023. URL https://huggingface.co/datasets/FreedomIntelligence/alpaca-gpt4-deutsch.

Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.

Gat, I., Remez, T., Shaul, N., Kreuk, F., Chen, R. T. Q., Synnaeve, G., Adi, Y., and Lipman, Y. Discrete flow matching. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Hu, L., Feng, Z., Wu, Y., Yuan, H., Zhao, Y., Qian, Y.-Y., Wang, B., Zhao, P., Jiang, D., Zhu, Y., Rosing, T., and Zhang, H. Jetspec: Breaking the scaling ceiling of speculative decoding with parallel tree drafting. *arXiv preprint arXiv:2606.18394*, 2026.

Huang, J., Zhang, Y., Zhang, Q., Lin, H., Xu, H., and Zhang, L. Domino: Decoupling causal modeling from autoregressive drafting in speculative decoding. *arXiv preprint arXiv:2605.29707*, 2026.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.

Lee, C., Yoo, J., Agarwal, M., Shah, S., Huang, J., Raghunathan, A., Hong, S., Boffi, N. M., and Kim, J. Flow map language models: One-step language modeling via continuous denoising. *arXiv preprint arXiv:2602.16813*, 2026.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, pp. 19274–19286, 2023.

Li, G., Fu, Z., Fang, M., Zhao, Q., Tang, M., Yuan, C., and Wang, J. Diffuspec: Unlocking diffusion language models for speculative decoding. *arXiv preprint arXiv:2510.02358*, 2025a.

Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024a.

Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024b.

Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025b.

Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

Lipman, Y., Havasi, M., Holderrieth, P., Shaul, N., Le, M., Karrer, B., Chen, R. T. Q., Lopez-Paz, D., Ben-Hamu, H., and Gat, I. Flow matching guide and code. *arXiv preprint arXiv:2412.06264*, 2024.

Liu, F., Li, X., Zhao, K., Gao, Y., Zhou, Z., Zhang, Z., Wang, Z., Dou, W., Zhong, S., and Tian, C. Dart: Diffusion-inspired speculative decoding for fast llm inference. *arXiv preprint arXiv:2601.19278*, 2026.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 932–949, 2024.

Nallapati, R., Zhou, B., dos Santos, C., Gülçehre, Ç., and Xiang, B. Abstractive text summarization using sequence-to-sequence RNNs and beyond. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pp. 280–290. Association for Computational Linguistics, 2016. doi: 10.18653/v1/K16-1028. URL https://aclanthology.org/K16-1028/.

Nathawani, D., Gitman, I., Majumdar, S., Bakhturina, E., Sunil Mahabaleshwarkar, A., Zhang, J., and Polak Scowcroft, J. Nemotron-Post-Training-Dataset-v1. NVIDIA dataset, version 1.0, July 2025. URL https://huggingface.co/datasets/nvidia/Nemotron-Post-Training-Dataset-v1.

Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

Oh, S., Cao, N., Kim, Y., Jung, H., Ahmad, H., Bae, S., and Yun, S.-Y. BASTION: Budget-aware speculative decoding with tree-structured block diffusion drafting. *arXiv preprint arXiv:2605.29727*, 2026. URL https://arxiv.org/abs/2605.29727.

OpenAssistant Contributors. OpenAssistant Conversations Dataset 2 (OASST2). Hugging Face dataset, 2023. URL https://huggingface.co/datasets/OpenAssistant/oasst2.

Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

Rajani, N., Tunstall, L., Beeching, E., Lambert, N., Rush, A. M., and Wolf, T. No robots. Hugging Face dataset, 2023. URL https://huggingface.co/datasets/HuggingFaceH4/no_robots.

Ringel, L. and Romano, Y. Accelerating speculative decoding with block diffusion draft trees. *arXiv preprint arXiv:2604.12989*, 2026.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. URL https://arxiv.org/abs/2112.10752.

Sadhukhan, R., Chen, J., Chen, Z., Tiwari, V., Lai, R., Shi, J., Yen, I. E.-H., May, A., Chen, T., and Chen, B. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. *International Conference on Learning Representations (ICLR)*, 2025.

Samragh, M., Kundu, A., Harrison, D., Nishu, K., Naik, D., Cho, M., and Farajtabar, M. Your llm knows the future: Uncovering its multi-token prediction potential. *arXiv preprint arXiv:2507.11851*, 2025.

Saxena, A. Prompt lookup decoding, 2023. https://github.com/apoorvumang/prompt-lookup-decoding/.

Shao, C. et al. Continuous autoregressive language models. *arXiv preprint arXiv:2510.27688*, 2025.

Shazeer, N. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

Stern, M., Shazeer, N., and Uszkoreit, J. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

Tiedemann, J. Parallel data, tools and interfaces in OPUS. In *Proceedings of the Eighth International Conference on Language Resources and Evaluation (LREC'12)*. European Language Resources Association, 2012. URL https://aclanthology.org/L12-1246/.

Wang, J. et al. Fudoki: Discrete flow-based unified understanding and generation via kinetic-optimal velocities. *arXiv preprint arXiv:2505.20147*, 2025.

A TRAINING DATA

ChainFlow is an efficient and adaptable speculative decoding method, and its adaptability is most visible in the speedups it delivers across domains and languages. To demonstrate this plasticity, we train on four languages: English, Turkish, German and French. We draw on multiple datasets per language, which together cover a wide spectrum of domains: mathematics, code, chat, multi-turn dialogue, retrieval-augmented generation, summarisation, translation and creative writing. Table 10 lists every source.

Table 10. Training sources by language and domain. Drafters are trained on English first, then adapted in sequence to Turkish, German and French without English replay. English totals 45,887 rows (16.6M tokens), while Turkish, German and French 8,000 each. Replay stages add 2,088 Turkish rows (20.7%). Rows are prompts, and training responses are the target's own greedy continuations.

Language	Source	Domain	Rows
English	Code Alpaca (Chaudhary, 2023)	Code	5,000
	Dolly (Conover et al., 2023)	Instruction	5,000
	Nemotron v1, math (Nathawani et al., 2025)	Math	5,000
	Nemotron v1, STEM (Nathawani et al., 2025)	Science	5,000
	GSM8K (Cobbe et al., 2021)	Math	5,000
	UltraChat (Ding et al., 2023)	Dialogue	5,000
	No Robots (Rajani et al., 2023)	Instruction	4,000
	WritingPrompts (Fan et al., 2018)	Creative writing	3,960
	CNN/DailyMail (Nallapati et al., 2016)	Summarisation	4,000
	OPUS Books, EN→FR (Tiedemann, 2012)	Translation	3,927
Turkish	InstrucTurca (Altinok, 2024)	Instruction	5,278
	Function calling (Ataşoğlu, n.d.)	Function calling	1,600
	Tool calling (Ataşoğlu, 2025)	Tool calling	481
	Multi-turn dialogue (Bayram, 2026)	Dialogue	641
German	Alpaca-GPT4-Deutsch (FreedomIntelligence, 2023)	Instruction	4,794
	GPT-4 Self-Instruct German (CausalLM, n.d.)	Instruction	1,692
	GermanRAG (Auras et al., n.d.)	RAG	846
	OASST2 (de) (OpenAssistant Contributors, 2023)	Dialogue	668
French	French-Alpaca-Instruct	Instruction	4,956
	French Instruct ShareGPT	Dialogue	2,623
	OASST2 (fr) (OpenAssistant Contributors, 2023)	Dialogue	421